

SECOND TECHNICAL TASK - CONTROLLED EXECUTION

Task 2

Local Toolchain and Container Foundation

The execution guide for creating a reproducible local development baseline while keeping all fraud, transaction-processing, ML and AI work explicitly deferred.

Prepared for: Ritwik Dhull

Version: 1.0

Date: 9 August 2026

Milestone: Four-week production-readiness demonstration

Budget boundary: Free/local only



1. Decision and Task Boundary

TASK 2 OUTCOME

From a clean checkout, a developer can bootstrap, start, verify and stop a version-controlled local platform shell consisting of a FastAPI service, a React/TypeScript web shell, PostgreSQL and Redis. The result is infrastructure proof, not a fraud-detection feature.

1.1 Included and excluded

Included now	Explicitly excluded now
Python 3.12 project managed with uv	ML libraries, notebooks, models or training
FastAPI shell with `/health` and `/ready` only	Transaction, scoring, alert or admin APIs
Node.js 22 LTS, npm and locked dependencies	Product screens, analyst workflows or charts
React + TypeScript + Vite operational shell	Fraud rules, probability scores or decisions
Docker Compose: API, web, PostgreSQL and Redis	Redis Streams, workers or consumer groups
Formatting, linting, typing and smoke tests	Database business schemas or migrations
Safe environment template and local run commands	ULB dataset ingestion or profiling

1.2 Fixed technical decisions

Area	Decision	Control
Repository	Use the Task 1 monorepo foundation	Preserve existing work and report dirty files
Backend	Python 3.12 + uv + FastAPI	Lock dependencies; no ML/data stack
Frontend	Node.js 22 LTS + npm + React/TS/Vite	Commit `package-lock.json`
Local services	Docker Compose	No `latest`; explicit compatible image tags
Data stores	PostgreSQL and Redis	Health only; no business state yet
Verification	One root command: `make verify`	Fail fast and return non-zero on failure

NO SCOPE CREEP

Codex must stop after the platform shell passes Task 2 verification. A working database or Redis container does not authorize schema design, transaction persistence, stream creation, scoring, ML, AI or dashboard features.

2. Manual Part - Preflight Only

Codex performs the checks first. You act only when Codex proves that a human-controlled prerequisite is missing or blocked.

ID	Potential manual action	Needed only when	Blocking
M1	Open Codex at the repository root	Codex is in the wrong folder	Yes
M2	Install/start Docker Desktop and enable Compose	Docker/Compose check fails	Yes
M3	Install Git, Python 3.12, uv or Node.js 22 LTS	Required host tool is absent/incompatible	Yes
M4	Approve package/image downloads	Environment requires network approval	Yes
M5	Resolve an occupied localhost port	Approved default cannot bind	Yes
M6	Review the plan and later the full diff	Before edits and before commit	Yes
M7	Approve commit and push	After all checks pass	No

2.1 Preflight commands Codex should run

```
git rev-parse --show-toplevel
git status --short
git --version
docker --version
docker compose version
python --version
uv --version
node --version
npm --version
```

2.2 Manual security rules

- Never paste passwords, tokens, private keys, database passwords or other secrets into the Codex conversation.
- Use only generated local development values. Real payment credentials, PAN, CVV, PIN and cardholder data are prohibited.
- Do not grant Codex access outside the repository merely to solve a local setup issue.
- If Docker requires a UI action or elevated installation, complete it yourself after reading Codex's exact request.
- If a port is occupied, identify the owning process before closing it; prefer changing the project port if the process is legitimate.

EXPECTED OWNER EFFORT

If the tools are already installed and Docker is running, your only blocking actions should be approving the Codex plan and reviewing the resulting diff. Codex should not delegate routine file creation, configuration or verification to you.

3. Codex Part A - Read-Only Plan

Paste this in Codex from the repository root using Plan mode. This authorizes inspection only.

TASK

Plan only Task 2: local toolchain and container foundation. ML, AI, fraud logic and transaction processing remain deferred.

INSPECT FIRST

Read root `AGENTS.md`, `README.md`, `.gitignore`, `.env.example`, all relevant planning files, and the existing repository tree. Confirm the Git root and report pre-existing/dirty changes. Run the Task 2 host-tool preflight commands.

PROPOSE

1. Python 3.12 + uv backend shell using FastAPI.
2. `GET /health` for process liveness and `GET /ready` for PostgreSQL/Redis reachability only.
3. Node.js 22 LTS + npm React/TypeScript/Vite shell.
4. Dockerfiles and one Compose stack for API, web, PostgreSQL and Redis.
5. Explicit compatible image tags, health checks, named volumes and localhost-only ports.
6. Root Makefile commands: `bootstrap`, `up`, `down`, `logs`, `ps`, `verify` and `clean` if safe.
7. Ruff, mypy and pytest for backend; ESLint, Prettier, TypeScript and Vitest for web.
8. Placeholder-only `.env.example`, with `.env` and local state excluded from Git.
9. Exact smoke tests, dependency locks and documentation updates.

DO NOT IMPLEMENT

No transaction routes, schemas, migrations, Redis Streams, workers, authentication, product UI, dataset work, fraud rules, scoring, ML, AI, pandas, NumPy, scikit-learn, River or MLflow.

RETURN

Proposed files, dependency choices, service topology, port plan, commands, tests, acceptance checks, risks, manual blockers and rollback approach. Do not edit or install anything. Wait for approval.

3.1 Codex must challenge these conditions

- A dirty worktree with changes that overlap Task 2 files.
- Missing or contradictory root `AGENTS.md` instructions.
- A proposed dependency or service that is unnecessary for the exit criteria.
- Unpinned container images, uncommitted dependency locks or hard-coded credentials.
- A readiness check that mutates business state or silently reports healthy when dependencies are unavailable.
- A host-specific design that prevents a second developer from reproducing the setup.

PLAN QUALITY GATE

The plan is acceptable only if every file and dependency serves a named exit criterion. Anything beyond the operational shell must be removed or explicitly deferred.

4. Manual Blocker and Approval Protocol

The Task 1 protocol remains mandatory. Codex must not convert a human-only action into an assumption or conceal it inside a status paragraph.

MANUAL ACTION REQUIRED - [short title]

Status: BLOCKING or NON-BLOCKING

Why it is needed: [precise reason tied to Task 2]

What I verified: [commands/evidence already checked]

Your exact action: [numbered steps or exact UI path]

Expected confirmation: [safe evidence or reply]

Safety note: [credential, admin or destructive-action warning]

Codex state: Paused before [operation]. I will resume from [step].

Reply with: `Done - [confirmation]` or describe the error.

4.1 Trigger-to-response map

Observed blocker	Codex response
Docker daemon unavailable	Report exact failed check; ask owner to start/install Docker
Tool version missing or incompatible	Name required version and official install route; wait
Network download denied	List exact packages/images and purpose; request approval
Port collision	Report port and safe inspection command; ask owner to choose change/stop
Existing changes overlap	Identify files; ask whether to preserve, integrate or postpone
Secret value required	Ask owner to configure it outside chat; never request its value
Commit/push/merge required	Stop and request explicit approval

4.2 Owner approval checklist

- The plan contains exactly four local services: API, web, PostgreSQL and Redis.
- Only liveness/readiness and a neutral web shell are implemented.
- Dependencies and container images are explicit and locked/pinned; no `latest` tag.
- No real secrets or payment data are required.
- The verify command covers format, lint, type, tests, build, Compose validation and smoke checks.
- The plan contains no ML/AI, transaction processing, schemas, streams, fraud logic or product feature work.

If correct:

`Approved. Implement Task 2 exactly as planned. Stop after Task 2 verification and handoff.`

If incorrect:

`Revise the Task 2 plan with these corrections: [list]. Remain read-only and wait again.`

5. Required Task 2 Architecture

The implementation may vary in file naming only where the Task 1 repository already defines an equivalent convention.

Component	Minimum responsibility	Forbidden responsibility
`apps/api`	FastAPI app, settings, `/health`, `/ready`, tests	Transactions, scoring, users or business rules
`apps/web`	React/TS/Vite neutral operational shell and tests	Dashboard, alerts, analyst workflow or fraud display
PostgreSQL	Starts, persists local volume, passes health check	Fraud tables, migrations or seed data
Redis	Starts, persists/configures safely, passes ping check	Streams, consumers, velocity state or queues
Compose	Build/start/stop the four services predictably	Production orchestration or cloud deployment
Root tooling	One command surface and documented prerequisites	Hidden machine-specific setup

5.1 Health semantics

Endpoint/check	Pass condition	Failure condition
API `/health`	Process event loop responds	API process unavailable
API `/ready`	Configured PostgreSQL and Redis checks succeed	Either dependency cannot be reached
PostgreSQL health	Server accepts readiness probe	Starting, unavailable or invalid config
Redis health	Authenticated/configured ping succeeds	Unavailable or invalid config
Web health	Static/dev server responds and production build succeeds	Server/build unavailable

5.2 Environment contract

- `.env.example` contains names and safe development placeholders only; `.env` remains untracked.
- Ports bind to `127.0.0.1` and are configurable to avoid exposing local services on the LAN.
- Service names, container names, volumes and networks use a project-specific prefix.
- PostgreSQL and Redis credentials/configuration are development-only and visibly labelled as such.
- Readiness uses short timeouts and returns a non-success status when dependencies fail.
- No endpoint returns credential values, connection strings or unnecessary environment details.

VERSION CONTROL RULE

Commit lockfiles and explicit image tags. Do not use floating `latest` images. Record the versions Codex actually verified instead of copying assumed versions from this guide.

6. Codex Part B - Implementation Prompt

Use only after the read-only plan is approved.

AUTHORIZATION

Implement only the approved Task 2 local toolchain and container foundation.

EXECUTION RULES

- Re-read `AGENTS.md`; preserve all user changes.
- Apply the approved structure and smallest sufficient dependencies.
- Lock Python and npm dependencies and pin explicit compatible container image tags.
- Add only `/health`, `/ready` and a neutral React shell.
- Add Compose health checks, named volumes, localhost-only ports and safe env defaults.
- Add documented root commands and tests.
- If installation, Docker UI, network approval, secrets, port decisions or conflicting edits require me, use `MANUAL ACTION REQUIRED` and pause that work.
- Do not commit, push or merge without separate approval.

VERIFY

Run format/lint/type/tests/build, validate Compose, build/start the stack, wait for health, request the API endpoints and web shell, inspect logs, stop/restart once, confirm persistence/health semantics, then shut down cleanly. Report every exact command and outcome; never claim an unrun check.

STOP

Do not begin transaction contracts, schemas, Redis Streams, workers, fraud logic, dataset work, ML, AI or the next task. Return the Task 2 handoff format from this guide.

6.1 Preferred command contract

Command	Required behaviour
`make bootstrap`	Install locked dev dependencies without starting services
`make up`	Build and start the four-service local stack
`make ps`	Show service and health state
`make logs`	Show bounded/recent service logs
`make verify`	Run all static checks, tests, builds, Compose validation and smoke checks
`make down`	Stop services without deleting named volumes
`make clean`	Remove generated caches/builds only; never delete data volumes by default

DESTRUCTIVE SAFETY

Volume deletion must be a separate, clearly named command requiring explicit owner approval. `make clean` and `make down` must not destroy PostgreSQL or Redis volumes.

7. Verification and Acceptance Gate

7.1 Required verification layers

Layer	Minimum check	Evidence
Repository	Git status, ignored secrets/state, diff review	No unintended/untracked sensitive files
Backend	Ruff, mypy, pytest	All pass; health/readiness tests included
Frontend	Prettier, ESLint, TypeScript, Vitest, build	All pass; shell renders
Containers	Compose config/build/start/health	Four services healthy
Smoke	Request API endpoints and web root	Expected status and minimal body
Failure	Pause one dependency and test `/ready`	Readiness fails visibly; `/health` remains valid
Restart	Restart stack without rebuild	Returns to healthy state
Logs	Inspect for secrets/errors	No secrets or unexplained errors
Scope	Search dependencies and source	No forbidden ML/business implementation

7.2 Binary exit criteria

1. A clean clone can follow the documented bootstrap path without undocumented local files.
2. API, web, PostgreSQL and Redis build and become healthy through the approved Compose stack.
3. `/health` reports process liveness; `/ready` truthfully represents PostgreSQL and Redis reachability.
4. The neutral React shell loads and its production build succeeds.
5. The unified verification command passes every configured check.
6. Dependency locks, explicit image tags and safe environment templates are committed candidates.
7. No real secret, payment data, raw dataset or generated service state is tracked.
8. No transaction processing, business schema, Redis Stream, fraud rule, model, ML or AI code exists.
9. Codex reports exact commands, outcomes, deviations and pending manual actions.

FAIL MEANS NOT COMPLETE

Task 2 is not complete when checks are skipped, health is inferred from container startup, readiness lies about dependencies, or a machine-specific workaround is undocumented. Mark the task partial or blocked and identify the exact next action.

8. Required Codex Final Handoff

TASK 2 STATUS

`COMPLETED` | `PARTIALLY COMPLETED` | `BLOCKED`

Outcome

[One sentence describing the operational platform shell.]

Files created or changed

[Grouped list with purpose; separate preserved pre-existing changes.]

Dependency and image baseline

[Actual locked versions/tags and why each direct dependency exists.]

Verification performed

[Exact commands, exit results, endpoint results and service health.]

Scope confirmation

- No transaction/business implementation.
- No database business schema or Redis Stream/worker.
- No fraud logic, dataset processing, ML or AI.
- No commit/push/merge unless separately approved.

Risks, deviations or skipped checks

[List with impact; or `None`.]

PENDING MANUAL ACTIONS - BLOCKING

[Exact action, evidence needed and resume point; or `None`.]

PENDING MANUAL ACTIONS - NON-BLOCKING

[Diff review, commit/push, team setup or follow-up; or `None`.]

NEXT TASK

[Name only. State that it has not started.]

8.1 Manual actions after Codex finishes

Manual action	When
Review all changed files and the complete diff	Always
Confirm no secrets, datasets, volumes or unrelated files are staged	Always
Open the web shell and API endpoints on your machine	Before acceptance
Resolve blocking manual actions and ask Codex to rerun failed checks	If listed
Approve commit and push	Only after binary exit criteria pass
Record non-blocking actions in the project ledger	Before their due date

TASK 2 ENDS HERE

The next task is the transaction event contract and ingestion boundary. It must not begin until Task 2 is accepted and separately authorized. ML and AI remain deferred beyond that next non-ML foundation step.

9. Working Rules and Reference Basis

9.1 Codex operating rules

- Treat root `AGENTS.md` as durable repository guidance and keep task-specific detail in the planning documents.
- Use the pattern goal, context, constraints and done-when criteria for every implementation request.
- Plan before multi-step edits, validate the result, inspect the diff and report what actually ran.
- Keep permissions limited to the real repository and pause before external, destructive or human-controlled actions.
- Update `AGENTS.md` only when Task 2 reveals a durable rule that will prevent repeated mistakes.

9.2 Official OpenAI guidance

- Codex best practices: <https://learn.chatgpt.com/guides/best-practices>
- Custom instructions with AGENTS.md: <https://learn.chatgpt.com/docs/agent-configuration/agents-md>

9.3 Next-task decision

NEXT AFTER ACCEPTANCE

Task 3 should define the versioned credit-card authorization event contract, validation/quarantine rules, idempotency key and ingestion API boundary using synthetic fixtures only. It should still exclude ML, AI and fraud scoring.

OWNER-CONTROL PRINCIPLE

Codex performs the maximum safe implementation and verification. Ritwik retains environment installation/UI actions, credentials, unresolved product decisions, plan approval, diff acceptance, commit/push authority and permission to start the next task.